In the context of a Database Management System (DBMS), a script typically refers to a SQL script — a structured file or sequence of SQL commands used to automate tasks and manage database operations

What Is a SQL Script?

A SQL script is a collection of SQL statements written in a file or executed as a batch. These statements can include:

- **DDL (Data Definition Language):** CREATE, ALTER, DROP — for defining or modifying database structures.
- **DML (Data Manipulation Language):** INSERT, UPDATE, DELETE — for manipulating data.
- **DCL (Data Control Language):** GRANT, REVOKE — for managing access permissions.
- **TCL (Transaction Control Language):** COMMIT, ROLLBACK, SAVEPOINT — for managing transactions.

These scripts are used to:

- Automate repetitive tasks
- Ensure consistency across environments
- Facilitate database migrations and updates
- Perform backups or data transformations

Where Are Scripts Used?

SQL scripts are commonly executed through:

- Command-line tools (e.g., mysql, psql)

- Database IDEs (e.g., MySQL Workbench, SQL Server Management Studio)
- Web-based interfaces
- Embedded in application code for deployment or initialization

How Do You Pronounce "SQL"?

There are two widely accepted pronunciations:

Pronunciation	Phonetic	Description
S-Q-L	/ˌɛsˌkjuːˈɛl/	Pronouncing each letter individually: "ess-cue-ell"
Sequel	/ˈsiːkwəl/	Pronounced like the word "sequel"

Why the Confusion?

- Originally, SQL was called SEQUEL (Structured English Query Language), developed by IBM in the 1970s.
- Due to a trademark issue, the name was changed to SQL — but the pronunciation "sequel" stuck around.
- Even today, both pronunciations are correct and commonly used. Some companies and products prefer one over the other:
    - MySQL officially prefers "My Ess Cue Ell" but accepts "My Sequel" too.

So whether you say "S-Q-L" or "sequel," you're in good company. It often comes down to personal or regional preference.

DDL – Data Definition Language

DDL is used to define and manage the structure of database objects like tables, schemas, and indexes.

- **Purpose: Create, modify, or remove database structures.**
- **Common Commands:**
    - CREATE – Make new tables or objects
    - ALTER – Modify existing structures
    - DROP – Delete objects
    - TRUNCATE – Remove all data from a table

Key Traits:

- Changes are auto-committed (cannot be rolled back).
- Focuses on the schema of the database.

Note. Rolling back DDL (Data Definition Language) statements—like CREATE, ALTER, DROP, or TRUNCATE—depends heavily on the database system you're using. Here's the scoop:

🛠️ Exceptions & Workarounds by Database

Database System	Can Rollback DDL?	Notes
SQL Server	✅ Sometimes	DDL inside a BEGIN TRANSACTION block can be rolled back

Database System	Can Rollback DDL?	Notes
Oracle	❌ Usually not	DDL causes implicit commit before and after execution.
PostgreSQL	✅ Yes	Supports transactional DDL—can rollback if inside a transaction.
MySQL	❌ Mostly not	DDL is auto-committed; rollback not supported for most DDL.

General Rule: DDL = Auto-Commit

Most databases automatically commit DDL statements, which means:

- Once executed, the change is immediately saved.
- You cannot roll it back using a standard ROLLBACK command

Creating database

```
CREATE DATABASE IF NOT EXISTS testDB
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;
```

Notes

- **IF NOT EXISTS** prevents errors if the database already exists.

- **utf8mb4** is recommended for full Unicode support (including emojis).

What if utf8mb4/collate is omitted?

If you don’ t specify utf8mb4 when creating your MySQL database or tables, MySQL will use its default character set and collation, which might be something like latin1 or utf8 depending on your server configuration. Here's what that means:

⚠️ **Consequences of Not Using utf8mb4**

1. Limited Unicode Support

- utf8 in MySQL only supports up to 3 bytes per character, which excludes many characters like:
    - Emojis (😊, 🚀, ❤️)
    - Some Asian scripts and rare symbols
- utf8mb4 supports 4 bytes, covering all Unicode characters.

2. Data Loss or Errors

- If you try to insert unsupported characters (like emojis) into a utf8 column:
    - You might get an error like Incorrect string value.
    - Or worse, the character might be silently truncated or corrupted.

3. Inconsistent Behavior

- If your app or frontend expects full Unicode support but your DB doesn't provide it, you’ ll see:
    - Broken characters (□)
    - Failed inserts or updates
    - Search and sorting issues

✅ Why utf8mb4 Is Better

Feature	utf8 (MySQL)	utf8mb4
Max bytes per char	3	4
Emoji support	❌ No	✅ Yes
Full Unicode support	❌ Partial	✅ Full
Recommended for new apps	❌ No	✅ Yes

🔤 What Is a CHARACTER SET?

A character set defines how text is stored—specifically, how characters are encoded into bytes.

Examples:

- latin1: Supports Western European characters only.
- utf8: Supports many characters but not emojis or some Asian scripts.
- utf8mb4: Supports all Unicode characters, including emojis, rare symbols, and multilingual text.

Analogy:

Think of a character set like the alphabet your database understands. If you use a limited alphabet, it won’ t recognize certain characters.

🔡 What Is a COLLATE?

A collation defines how text is compared and sorted—like whether "A" equals "a", or how "é" compares to "e".

Examples:

- utf8mb4_general_ci: Case-insensitive, fast, but less accurate for some languages.
- utf8mb4_unicode_ci: Case-insensitive and linguistically accurate (better for international apps).
- utf8mb4_bin: Case-sensitive and compares based on binary values.

Analogy:

Collation is like the rules of grammar for sorting and comparing words. Different languages have different rules, and collations help your database follow them.

🧠 **Quick Summary**

Term	What It Controls	Example Value	Why It Matters
CHARACTER SET	How text is stored (encoding)	utf8mb4	Ensures full Unicode support
COLLATE	How text is compared/sorted	utf8mb4_unicode_ci	Affects search, sorting, comparisons

Delete database

```
DROP DATABASE IF EXISTS your_database_name;
```

Explanation:

- DROP DATABASE: Deletes the entire database, including all tables, data, views, procedures, etc.
- IF EXISTS: Prevents an error if the database doesn't exist.

Important Reminders

- This action **cannot be undone**.
- Make sure you’ ve backed up any important data.
- All users and permissions associated with the database will be lost.

What are Keys in DBMS?

KEYS in DBMS is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table). They allow you to find the relation between two tables. Keys help you uniquely identify a row in a table by a combination of one or more columns in that table. Key is also helpful for finding unique record or row from the table. Database key is also helpful for finding unique record or row from the table.

Example:

Employee ID	FirstName	LastName
11	Andrew	Johnson
22	Tom	Wood
33	Alex	Hale

In the above-given example, employee ID is a primary key because it uniquely identifies an employee record. In this table, no other employee can have the same employee ID.

Why we need a Key?

Here are some reasons for using sql key in the DBMS system.

- Keys help you to identify any row of data in a table. In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys ensure that you can uniquely identify a table record despite these challenges.
- Allows you to establish a relationship between and identify the relation between tables
- Help you to enforce identity and integrity in the relationship.

Types of Keys in Database Management System

There are mainly seven different types of Keys in DBMS and each key has it' s different functionality:

- **Super Key** - A super key is a group of single or multiple keys which identifies rows in a table.
- **Primary Key** - is a column or group of columns in a table that uniquely identify every row in that table.
- **Candidate Key** - is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes.
- **Alternate Key** - is a column or group of columns in a table that uniquely identify every row in that table.
- **Foreign Key** - is a column that creates a relationship between two tables. The purpose of foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.
- **Compound Key** - has two or more attributes that allow you to uniquely recognize a specific record. It is possible that each column may not be unique within the database.

- **Composite Key** - An artificial key which aims to uniquely identify each record is called a surrogate key. These kinds of keys are unique because they are created when you don't have any natural primary keys.
- **Surrogate Key** - An artificial key which aims to uniquely identify each record is called a surrogate key. These kinds of keys are unique because they are created when you don't have any natural primary keys.

What is the Super key?

A superkey is a group of single or multiple keys which identifies rows in a table. A Super key may have additional attributes that are not needed for unique identification.

Example:

EmpSSN	EmpNum	Empname
9812345098	AB05	Shown
9876512345	AB06	Roslyn
199937890	AB07	James

In the above-given example, EmpSSN, EmpSSN+Empname, EmpNum, EmpNum+Empname, EmpNum+Empname+EmpSSN are superkeys.

What is a Primary Key?

PRIMARY KEY is a column or group of columns in a table that uniquely identify every row in that table. The Primary Key can't be a duplicate meaning the same value can't appear more than once in the table. A table cannot have more than one primary key.

Rules for defining Primary key:

- Two rows can't have the same primary key value
- It’ s best practice that every row to have a primary key value.
- The primary key field cannot be null.
- The value in a primary key column can never be modified or updated if any foreign key refers to that primary key.

Example:

In the following example, **StudID** is a Primary Key.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

What is the Alternate key?

ALTERNATE KEYS is a column or group of columns in a table that uniquely identify every row in that table. A table can have multiple choices for a primary key but only one can be set as the primary key. All the keys which are not primary key are called an Alternate Key.

Example:

In this table, StudID, Roll No, Email are qualified to become a primary key. But since StudID is the primary key, Roll No, Email becomes the alternative key.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com

What is a Candidate Key?

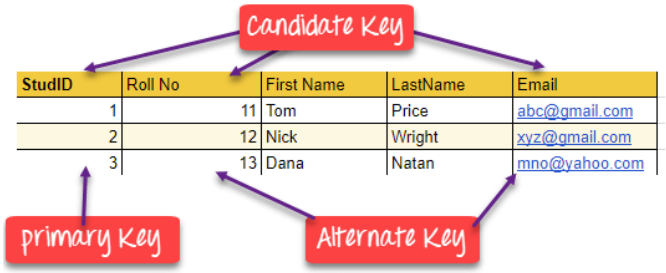
CANDIDATE KEY is a set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes. The Primary key should be selected from the candidate keys. Every table must have at least a single candidate key. A table can have multiple candidate keys but only a single primary key.

Properties of Candidate key:

- It must contain unique values
- Candidate key may have multiple attributes
- Must not contain null values
- It should contain minimum fields to ensure uniqueness
- Uniquely identify each record in a table

Example: In the given table Stud ID, Roll No, and email are candidate keys which help us to uniquely identify the student record in the table.

StudID	Roll No	First Name	LastName	Email
1	11	Tom	Price	abc@gmail.com
2	12	Nick	Wright	xyz@gmail.com
3	13	Dana	Natan	mno@yahoo.com



What is the Foreign key?

FOREIGN KEY is a column that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity. It acts as a cross-reference between two tables as it references the primary key of another table.

Example:

DeptCode	DeptName
001	Science
002	English
005	Computer

Teacher ID	Fname	Lname
B002	David	Warner
B017	Sara	Joseph
B009	Mike	Brunton

In this key in dbms example, we have two table, teacher and department in a school. However, there is no way to see which search work in which department.

In this table, adding the foreign key in Deptcode to the Teacher name, we can create a relationship between the two tables.

Teacher ID	DeptCode	Fname	Lname
B002	002	David	Warner
B017	002	Sara	Joseph
B009	001	Mike	Brunton

This concept is also known as Referential Integrity.

What is the Compound key?

COMPOUND KEY has two or more attributes that allow you to uniquely recognize a specific record. It is possible that each column may not be unique by itself within the database. However, when combined with the other column or columns the combination of composite keys become unique. The purpose of the compound key in database is to uniquely identify each record in the table.

Example:

OrderNo	PorductID	Product Name	Quantity
B005	JAP102459	Mouse	5
B005	DKT321573	USB	10
B005	OMG446789	LCD Monitor	20
B004	DKT321573	USB	15
B002	OMG446789	Laser Printer	3

In this example, OrderNo and ProductID can't be a primary key as it does not uniquely identify a record. However, a compound key of Order ID and Product ID could be used as it uniquely identified each record.

What is the Composite key?

COMPOSITE KEY is a combination of two or more columns that uniquely identify rows in a table. The combination of columns guarantees uniqueness, though

individually uniqueness is not guaranteed. Hence, they are combined to uniquely identify records in a table.

The difference between compound and the composite key is that any part of the compound key can be a foreign key, but the composite key may or maybe not a part of the foreign key.

A key made of two or more columns to uniquely identify a row.

example: PRIMARY KEY (order_id, product_id)

What is a Surrogate key?

SURROGATE KEYS is an artificial key which aims to uniquely identify each record is called a surrogate key. This kind of partial key in dbms is unique because it is created when you don't have any natural primary key. They do not lend any meaning to the data in the table. Surrogate key is usually an integer. A surrogate key is a value generated right before the record is inserted into a table.

Fname	Lastname	Start Time	End Time
Anne	Smith	09:00	18:00
Jack	Francis	08:00	17:00
Anna	McLean	11:00	20:00
Shown	Willam	14:00	23:00

Above, given example, shown shift timings of the different employee. In this example, a surrogate key is needed to uniquely identify each employee.

Surrogate keys in sql are allowed when:

- No property has the parameter of the primary key.
- In the table when the primary key is too big or complicated.

Difference Between Primary key & Foreign key

Primary Key	Foreign Key
Helps you to uniquely identify a record in the table.	It is a field in the table that is the primary key of another table.
Primary Key never accept null values.	A foreign key may accept multiple null values.
Primary key is a clustered index and data in the DBMS table are physically organized in the sequence of the clustered index.	A foreign key cannot automatically create an index, clustered or non-clustered. However, you can manually create an index on the foreign key.
You can have the single Primary key in a table.	You can have multiple foreign keys in a table.

Quick Notes

- Primary Key is the most important—it guarantees uniqueness and is often used for joins.
- Foreign Keys help maintain relationships between tables (e.g., orders linked to users).
- Unique Keys are great for fields like email or username.
- Indexes are not constraints but are essential for performance.

Common MySQL Constraints

Constraint Type	Description	Example
PRIMARY KEY	Uniquely identifies each row in a table	id INT PRIMARY KEY
FOREIGN KEY	Ensures referential integrity between tables	FOREIGN KEY (user_id) REFERENCES users(id)

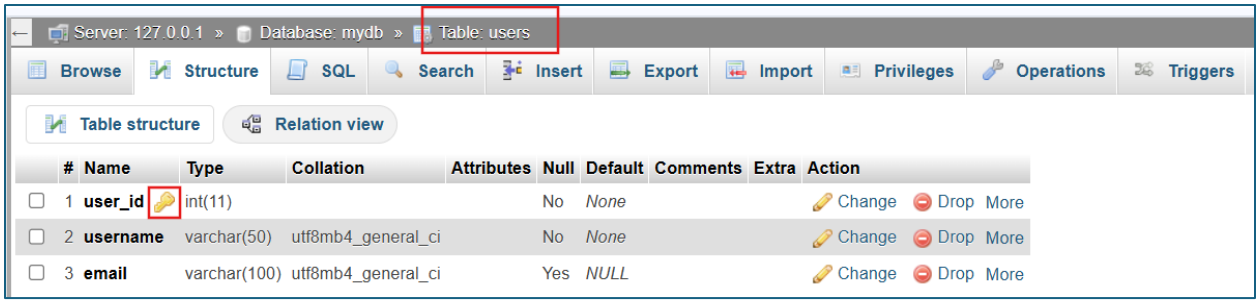
Constraint Type	Description	Example
UNIQUE	Ensures all values in a column are different	email VARCHAR(255) UNIQUE
NOT NULL	Prevents null values in a column	name VARCHAR(100) NOT NULL
CHECK	Ensures values meet a specific condition	CHECK (age >= 18)
DEFAULT	Sets a default value for a column	status VARCHAR(20) DEFAULT 'active'

Examples

👰 **Primary Key**

```
use mydb;
CREATE TABLE users (
  user_id INT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100)
);
```

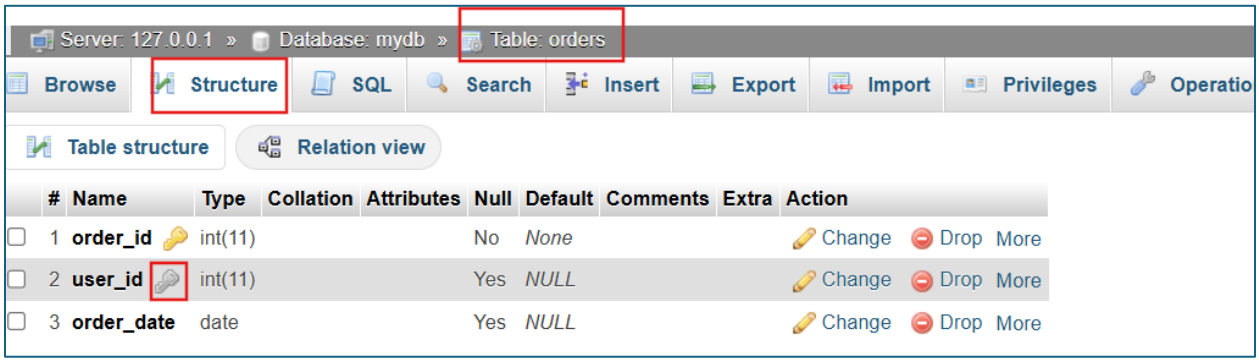
- **user_id uniquely identifies each user.**
- **Cannot be NULL.**

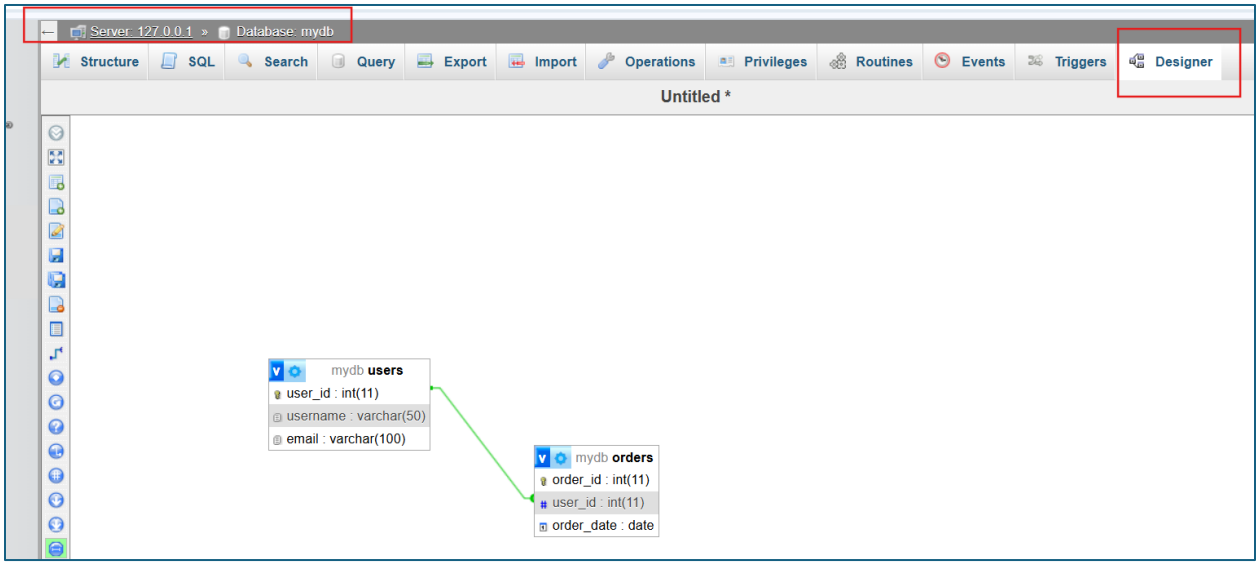


👨‍💼 Foreign Key

```
CREATE TABLE orders (
 order_id INT PRIMARY KEY,
 user_id INT,
 order_date DATE,
 FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

- user_id in orders links to user_id in users.
- Ensures that every order is tied to a valid user.





Unique Key

```
ALTER TABLE users
ADD CONSTRAINT unique_email UNIQUE (email);
```

- **Ensures no two users have the same email.**
- **Allows NULL, but only once.**

If you try to add with duplicate record you can see the error message below

```
#1062 - Duplicate entry '1' for key 'unique_email'
```

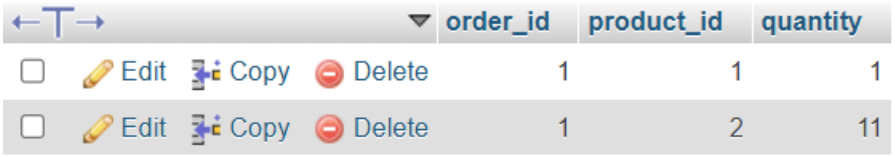
👨‍💼 Composite Key

```
CREATE TABLE order_items (
 order_id INT,
```

```
  product_id INT,
  quantity INT,
  PRIMARY KEY (order_id, product_id)
);
```

- **Combines order_id and product_id to uniquely identify each item in an order.**

Example



	order_id	product_id	quantity
Edit Copy Delete	1	1	1
Edit Copy Delete	1	2	11

You have two records, so if you add another record example order_id=1, product_id=2, then it will be an error

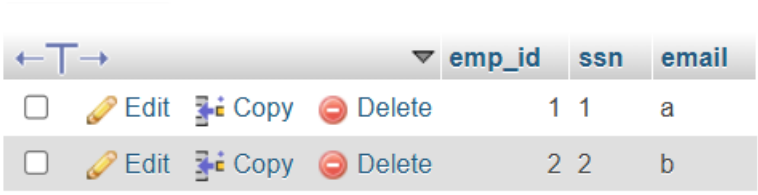
Note. If you will not put anything, then automatically it is zero, and still accepted basically

👨‍💼 **Candidate Key & Alternate Key**

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  ssn VARCHAR(20) UNIQUE,
  email VARCHAR(100) UNIQUE
);
```

- **ssn and email are candidate keys.**
- **Only one (e.g., emp_id) is chosen as the primary key.**
- **The others are alternate keys.**

Example



	emp_id	ssn	email
Edit Copy Delete	1	1	a
Edit Copy Delete	2	2	b

You have this record, so if you input 3,1,a, it will be an error, if you input 3, 1, c then still not ok, , if you input 3,2,d still not ok, but if you put 3, 4, d then its ok

👨‍💼 Index Key

```
CREATE INDEX idx_username ON users(username);
```

- **Speeds up searches on username.**
- **Doesn’ t enforce uniqueness or integrity.**

Issue the sql script to show the index

```
SHOW INDEX FROM users;
```

👨‍💼 Super Key

This is more of a concept than a syntax. Any combination of columns that uniquely identifies a row is a super key.

Example:

-- id alone is a super key

-- id + email is also a super key (redundant)

👩‍💼 Check

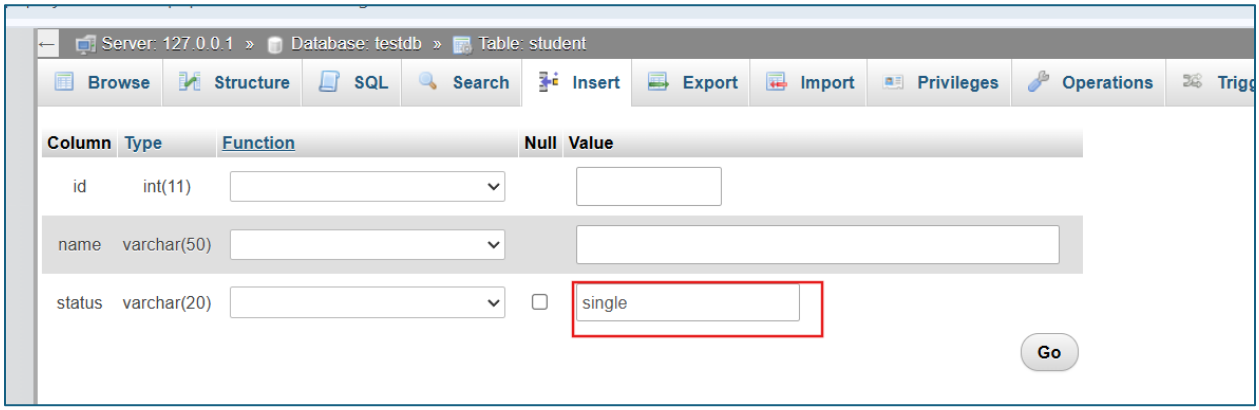
```
CREATE TABLE test (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    age INT,
    CHECK (age >= 5 AND age <= 100)
);
```

Here – Age field will only accept 5 to 100

```
INSERT INTO `test` (`id`, `name`, `age`) VALUES ('1', '1', '900')
MySQL said:
#4025 - CONSTRAINT `CONSTRAINT_1` failed for `testdb`.`test`
```

👩‍💼 DEFAULT

```
CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    status VARCHAR(20) DEFAULT 'single'
);
```



OTHER Commands

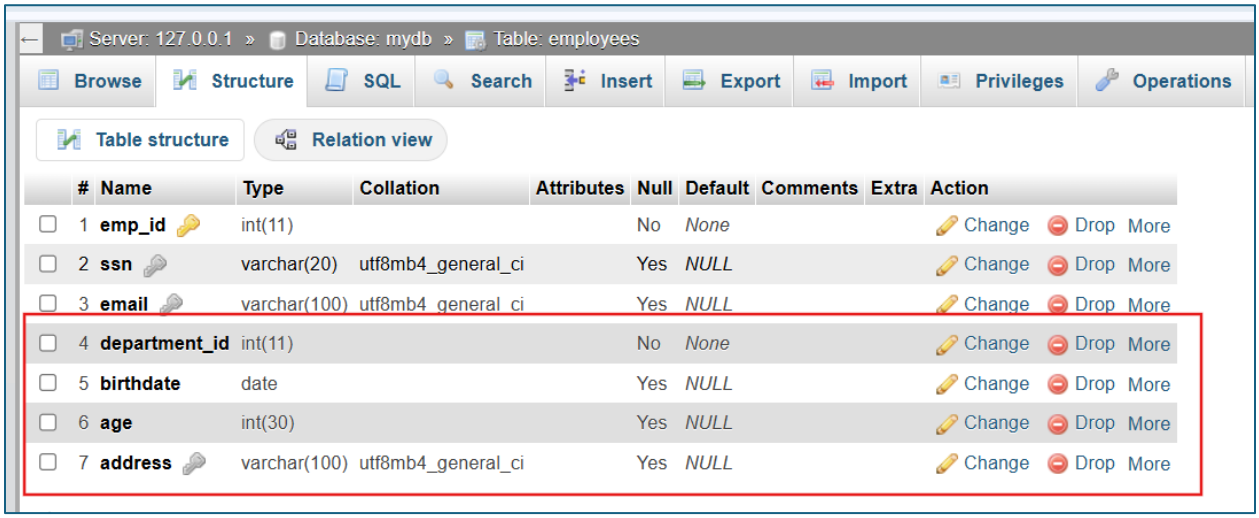
- **DESCRIBE table_name** - display table structure
- **SHOW CREATE TABLE table_name –** display the create table command

🛠️ Common ALTER TABLE Operations

Add a Column

```
ALTER TABLE employees
ADD COLUMN department_id INT NOT NULL,
ADD COLUMN birthdate DATE,
add column age int(30),
ADD COLUMN address VARCHAR(100) UNIQUE;
```

You can see in your table those added columns



Drop a Column

```
ALTER TABLE employees

DROP COLUMN address,

drop COLUMN age
```

Notes

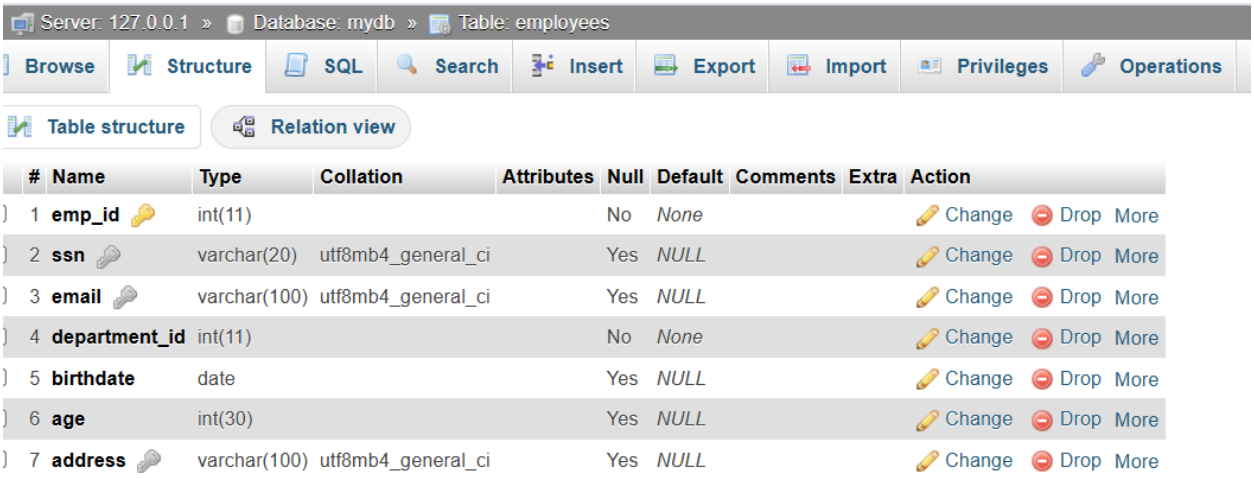
- All associated data in those columns will be permanently deleted.
- This action is irreversible unless you have a backup.
- Make sure the columns aren’ t used in views, triggers, or foreign key constraints.

3. Rename a Column

Put it back the deleted column

```
ALTER TABLE employees

add column age int(30),

ADD COLUMN address VARCHAR(100) UNIQUE;
```

Now you can see the fields

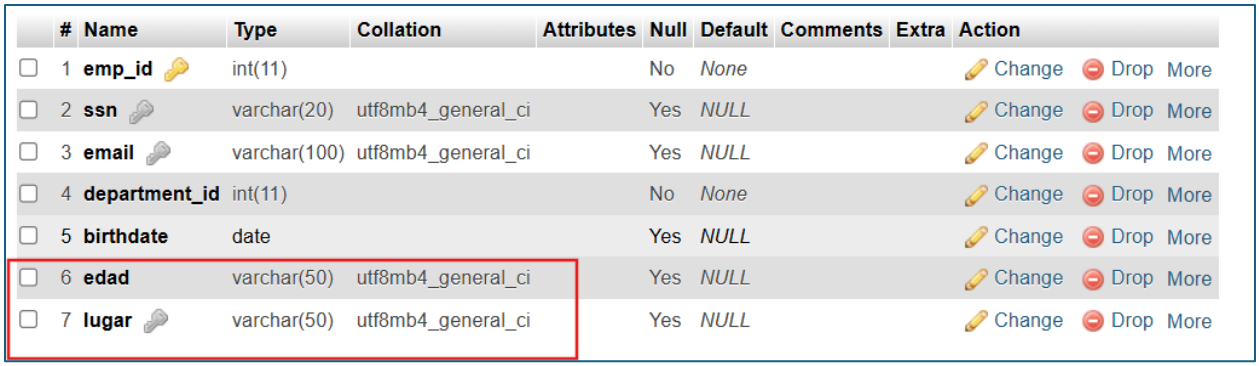


#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	emp_id	int(11)			No	None			Change Drop More
2	ssn	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	department_id	int(11)			No	None			Change Drop More
5	birthdate	date			Yes	NULL			Change Drop More
6	age	int(30)			Yes	NULL			Change Drop More
7	address	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Then do the rename

```
ALTER TABLE employees

CHANGE age edad varchar(50),

CHANGE address lugar varchar(50);
```

Your table structure now looks this way



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	emp_id	int(11)			No	None			Change Drop More
2	ssn	varchar(20)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	department_id	int(11)			No	None			Change Drop More
5	birthdate	date			Yes	NULL			Change Drop More
6	edad	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More
7	lugar	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Tips

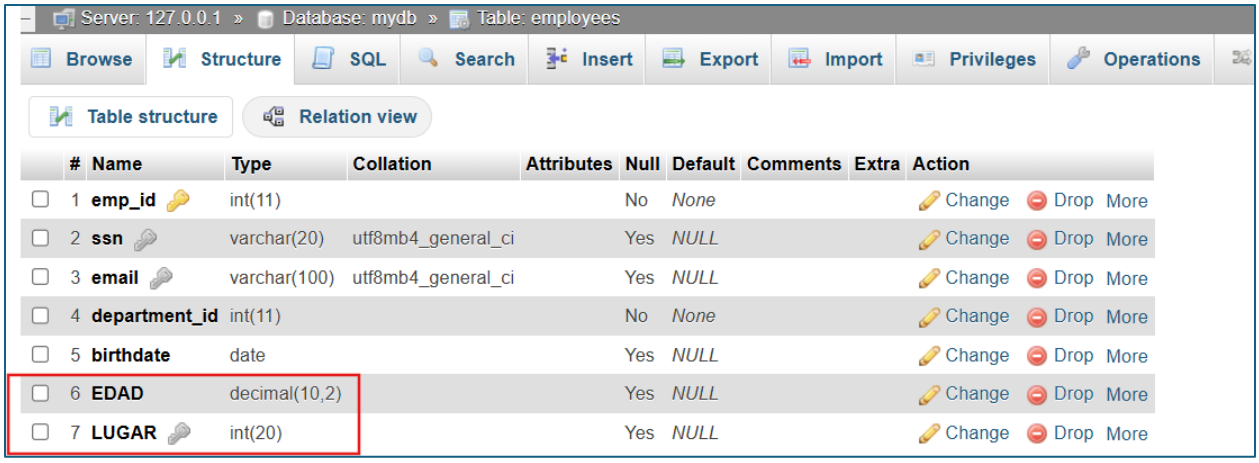
- You must specify the data type—even if it’ s not changing.
- If the column has constraints (e.g., NOT NULL, DEFAULT, etc.), include those too:

Note: RENAME COLUMN is supported in MySQL 8.0+. For older versions, use CHANGE COLUMN.

```
SELECT VERSION();

SHOW VARIABLES LIKE "%version%";
```

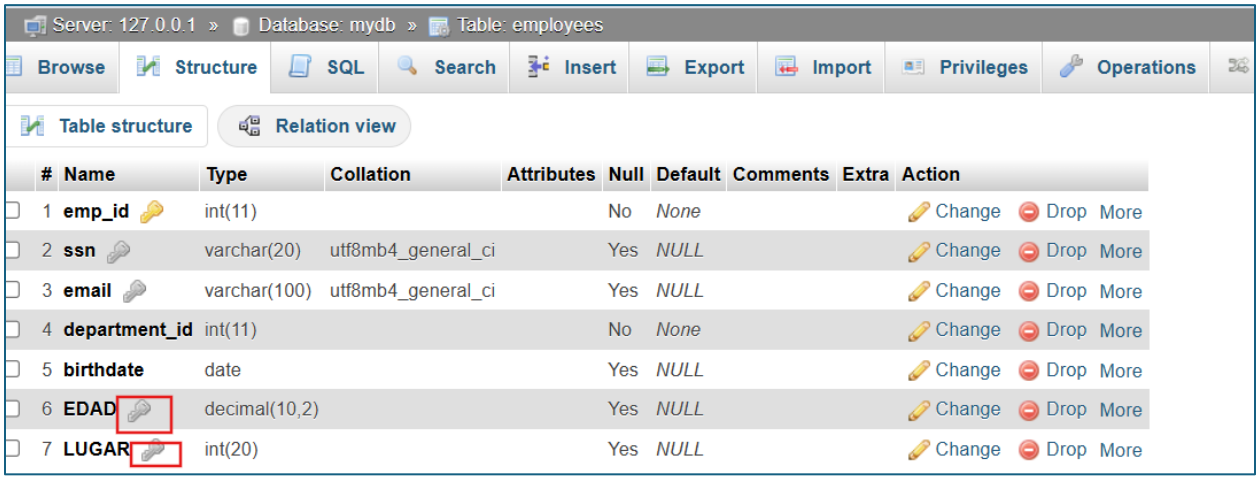
Change Column Data Type

```
ALTER TABLE employees

MODIFY COLUMN EDAD DECIMAL(10,2),

MODIFY COLUMN LUGAR INT(20)
```



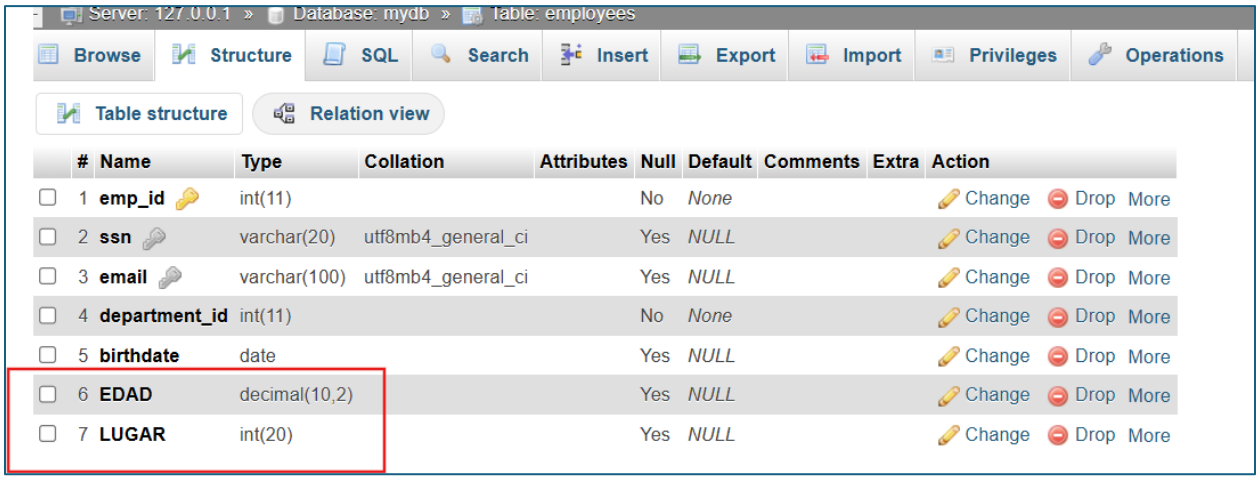
Add an Index

```
ALTER TABLE employees

ADD INDEX idx_EDAD (EDAD),

ADD INDEX idx_LUGAR (lugar);
```



Drop an Index

```
ALTER TABLE employees

drop INDEX idx_EDAD ,

drop INDEX idx_LUGAR ;
```



Rename the Table

```
rename table employees to emp
```

Add a Primary Key

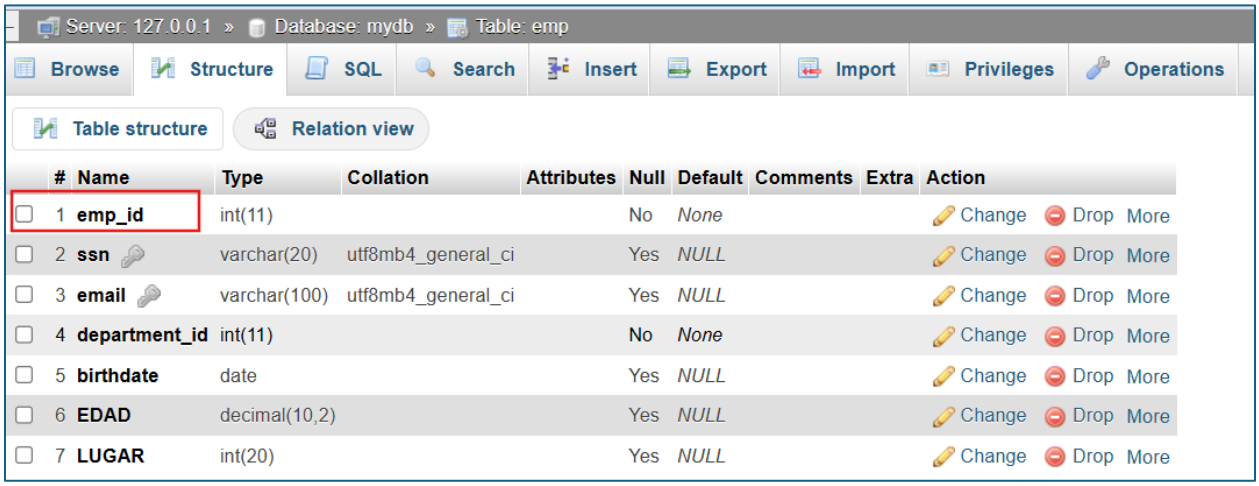
```
ALTER TABLE emp ADD PRIMARY KEY (emp_id);
```

if already have primary key then you can see this error

```
#1068 - Multiple primary key defined
```

Drop a Primary Key

```
ALTER TABLE emp DROP PRIMARY KEY;
```



Add a Foreign Key

```
ALTER TABLE orders

ADD CONSTRAINT fk_customer

FOREIGN KEY (user_id) REFERENCES emp(emp_id);
```

Create Table Using Another Table

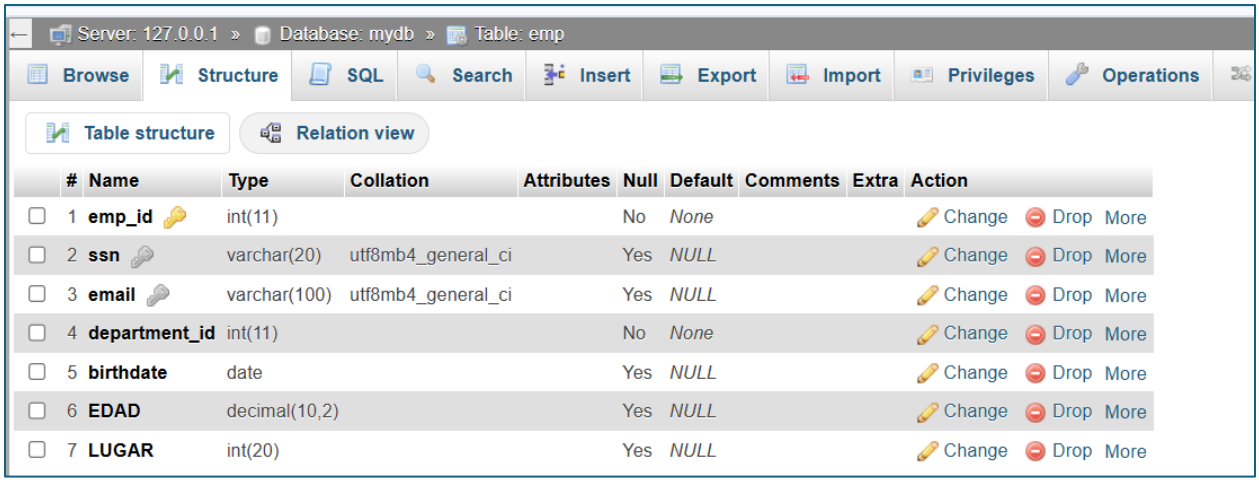
A copy of an existing table can also be created using CREATE TABLE.

The new table gets the same column definitions. All columns or specific columns can be selected.

If you create a new table using an existing table, the new table will be filled with the existing values from the old table.

Example

Source Table

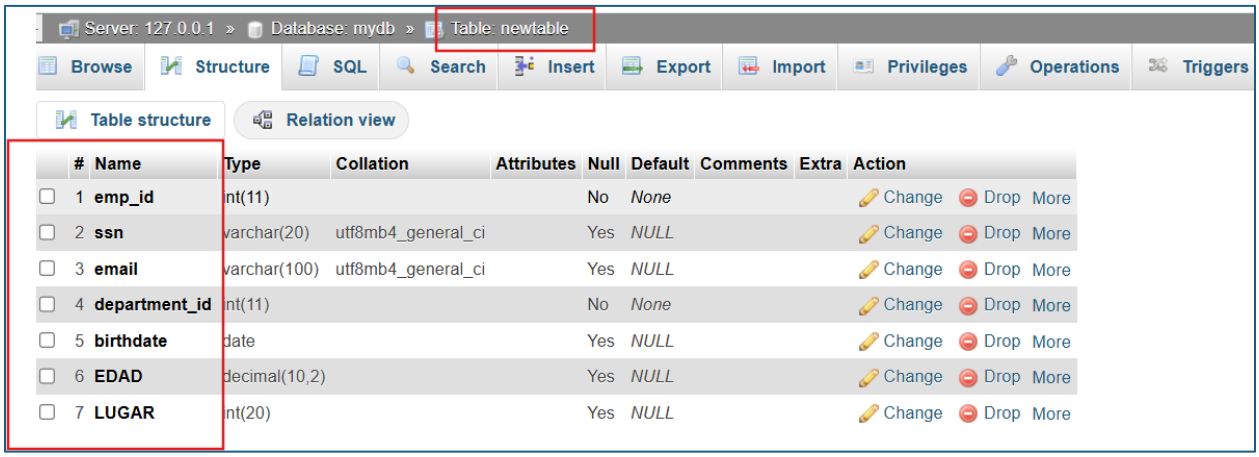


Destination Table

Issue the command below

```
CREATE TABLE new_table AS

SELECT * FROM emp;
```

Output



Note. It also copied the table records

What did you observe?

- ✅ Copies column definitions and data
- ❌ Does **not** copy indexes, constraints, or AUTO_INCREMENT

Create Table with Structure Only

```
CREATE TABLE new_table LIKE existing_table;
```

CREATE TABLE new_table LIKE emp;

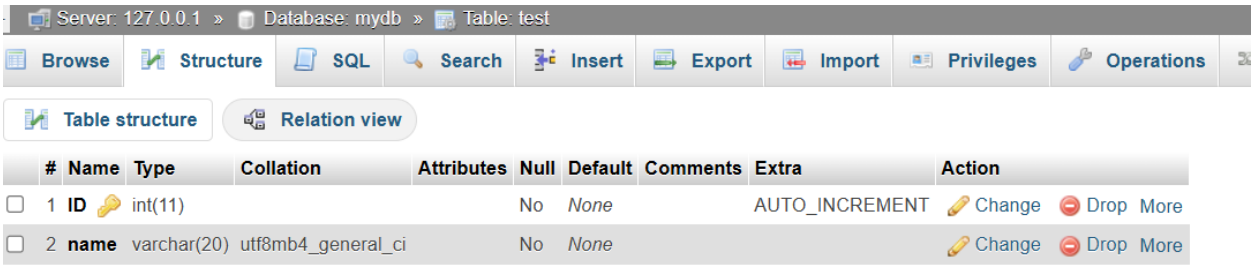
Note.

- ✅ Copies column definitions, indexes, constraints, AUTO_INCREMENT
- ❌ Does **not** copy data

If you need to copy everything including indexes and data, use CREATE TABLE LIKE followed by INSERT INTO

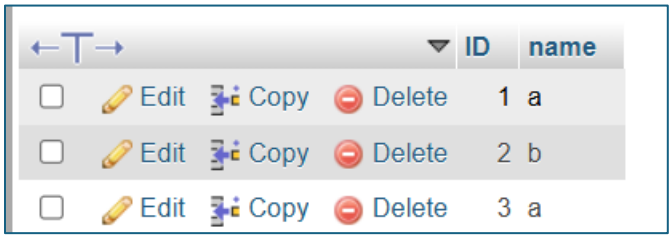
```
CREATE TABLE new_emp LIKE emp;

INSERT INTO new_emp SELECT * FROM emp;
```

Truncate Table



ID is primary key auto increment

Insert 3 records



Delete all record and insert again, the ID is still adding based on the last record.

What if you want to start the record clean, removing all increment? Use Truncate

TRUNCATE TABLE. This command removes all rows from a table very quickly and resets any AUTO_INCREMENT counters.

TRUNCATE test

Where test is the table name

Important Notes

- Irreversible: Unlike DELETE, you cannot undo a TRUNCATE unless you have a backup.
- Faster than DELETE: It doesn’ t log individual row deletions, making it much faster for large tables.
- Resets AUTO_INCREMENT: If your table has an AUTO_INCREMENT column, it resets to 1.
- Cannot truncate a table that is referenced by a foreign key constraint (unless you disable foreign key checks).

********** END **********